services that run very early in the boot sequence. The Linux `getrandom()` system call will block in this case until the OS has gathered enough entropy.

- When a virtual machine is repeatedly resumed from a snapshot it will have identical internal state until the OS re-seeds the random data generator. In some cases, this may result in identical or very similar output from the random device for a short time. While a genuine problem, you are unlikely to do a better job than the OS at detecting or handling this situation.

In short, trust the OS because most OS random data generators are well-designed and do a good job of generating unpredictable output. You should avoid the /dev/random device on Linux because it doesn't generate better quality output than /dev/urandom and may block your process for long periods of time. If you want to learn more about how operating systems generate random data securely, see chapter 9 of *Cryptography Engineering* by Niels Ferguson, Bruce Schneier, and Tadayoshi Kohno (Wiley, 2010).

For Natter, you'll use 160-bit token IDs generated with a `SecureRandom` object. First, generate 20 bytes of random data using the `nextBytes()` method. Then you can base64url-encode that to produce an URL-safe random string:

```
    private String randomId() {
        var bytes = new byte[20];    #A
        new SecureRandom().nextBytes(bytes);    #A
        return Base64url.encode(bytes);    #B
    }
```

#A Generate 20 bytes of random data from SecureRandom
#B Encode the result with URL-safe Base64 encoding to create a string

Listing 5.3 shows the complete `DatabaseTokenStore` implementation. After creating a random ID, you can serialize the token attributes into JSON and then insert the data into the `tokens` table using the Dalesbred library